

Example of path finding from City “Arad” to “Bucharest” in Romania map using UNIFORM COST SEARCH:

1. **Initialization:**
 - Start with node **Arad**.
 - Priority Queue: (0,Arad)(0, Arad)(0,Arad)
 - Explored set: { }
2. **Expand node Arad:**
 - Remove (0, Arad) from the priority queue.
 - Add Arad to the explored set.
 - Expand Arad to Zerind, Timisoara, and Sibiu with costs 75, 118, and 140 respectively.
 - Priority Queue: (75,Zerind),(118,Timisoara),(140,Sibiu)(75, Zerind), (118, Timisoara), (140, Sibiu)(75,Zerind),(118,Timisoara),(140,Sibiu)
 - Explored set: { Arad }
3. **Expand node Zerind:**
 - Remove (75, Zerind) from the priority queue.
 - Add Zerind to the explored set.
 - Expand Zerind to Oradea with cost 71.
 - Priority Queue: (118,Timisoara),(140,Sibiu),(146,Oradea)(118, Timisoara), (140, Sibiu), (146, Oradea)(118,Timisoara),(140,Sibiu),(146,Oradea)
 - Explored set: { Arad, Zerind }
4. **Expand node Timisoara:**
 - Remove (118, Timisoara) from the priority queue.
 - Add Timisoara to the explored set.
 - Expand Timisoara to Lugoj with cost 111.
 - Priority Queue: (140,Sibiu),(146,Oradea),(229,Lugoj)(140, Sibiu), (146, Oradea), (229, Lugoj)(140,Sibiu),(146,Oradea),(229,Lugoj)
 - Explored set: { Arad, Zerind, Timisoara }
5. **Expand node Sibiu:**
 - Remove (140, Sibiu) from the priority queue.
 - Add Sibiu to the explored set.
 - Expand Sibiu to Fagaras and Rimnicu Vilcea with costs 99 and 80 respectively.
 - Priority Queue:
(146,Oradea),(220,RimnicuVilcea),(229,Lugoj),(239,Fagaras)(146, Oradea), (220, Rimnicu Vilcea), (229, Lugoj), (239, Fagaras)(146,Oradea),(220,RimnicuVilcea),(229,Lugoj),(239,Fagaras)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu }
6. **Expand node Oradea:**
 - Remove (146, Oradea) from the priority queue.
 - Add Oradea to the explored set.
 - Expand Oradea to Sibiu with cost 151.
 - Priority Queue: (220,RimnicuVilcea),(229,Lugoj),(239,Fagaras)(220, Rimnicu Vilcea), (229, Lugoj), (239, Fagaras)(220,RimnicuVilcea),(229,Lugoj),(239,Fagaras) (Sibiu is already in the explored set and the new path cost 297 is not cheaper than 140, so it is not re-added)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea }
7. **Expand node Rimnicu Vilcea:**

- Remove (220, Rimnicu Vilcea) from the priority queue.
 - Add Rimnicu Vilcea to the explored set.
 - Expand Rimnicu Vilcea to Pitesti with cost 97.
 - Priority Queue: (229,Lugoj),(239,Fagaras),(317,Pitesti)(229, Lugoj), (239, Fagaras), (317, Pitesti)(229,Lugoj),(239,Fagaras),(317,Pitesti) (Cost from Arad to Pitesti via Sibiu and Rimnicu Vilcea is $220 + 97 = 317$)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu Vilcea }
- 8. Expand node Lugoj:**
- Remove (229, Lugoj) from the priority queue.
 - Add Lugoj to the explored set.
 - Expand Lugoj to Mehadia with cost 70.
 - Priority Queue: (239,Fagaras),(317,Pitesti),(299,Mehadia)(239, Fagaras), (317, Pitesti), (299, Mehadia)(239,Fagaras),(317,Pitesti),(299,Mehadia) (Cost from Arad to Mehadia via Timisoara and Lugoj is $229 + 70 = 299$)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu Vilcea, Lugoj }
- 9. Expand node Fagaras:**
- Remove (239, Fagaras) from the priority queue.
 - Add Fagaras to the explored set.
 - Expand Fagaras to Bucharest with cost 211.
 - Priority Queue: (299,Mehadia),(317,Pitesti),(450,Bucharest)(299, Mehadia), (317, Pitesti), (450, Bucharest)(299,Mehadia),(317,Pitesti),(450,Bucharest) (Cost from Arad to Bucharest via Sibiu and Fagaras is $239 + 211 = 450$)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu Vilcea, Lugoj, Fagaras }
- 10. Expand node Mehadia:**
- Remove (299, Mehadia) from the priority queue.
 - Add Mehadia to the explored set.
 - Expand Mehadia to Drobeta with cost 75.
 - Priority Queue: (317,Pitesti),(450,Bucharest),(374,Drobeta)(317, Pitesti), (450, Bucharest), (374, Drobeta)(317,Pitesti),(450,Bucharest),(374,Drobeta) (Cost from Arad to Drobeta via Timisoara, Lugoj, and Mehadia is $299 + 75 = 374$)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu Vilcea, Lugoj, Fagaras, Mehadia }
- 11. Expand node Pitesti:**
- Remove (317, Pitesti) from the priority queue.
 - Add Pitesti to the explored set.
 - Expand Pitesti to Bucharest with cost 101.
 - Priority Queue: (374,Drobeta),(450,Bucharest),(418,Bucharest)(374, Drobeta), (450, Bucharest), (418, Bucharest)(374,Drobeta),(450,Bucharest),(418,Bucharest) (Cost from Arad to Bucharest via Sibiu, Rimnicu Vilcea, and Pitesti is $317 + 101 = 418$)
 - Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu Vilcea, Lugoj, Fagaras, Mehadia, Pitesti }
- 12. Expand node Drobeta:**
- Remove (374, Drobeta) from the priority queue.
 - Add Drobeta to the explored set.
 - Expand Drobeta to Craiova with cost 120.

- Priority Queue: (418,Bucharest),(450,Bucharest),(494,Craiova)(418, Bucharest), (450, Bucharest), (494, Craiova)(418,Bucharest),(450,Bucharest),(494,Craiova) (Cost from Arad to Craiova via Timisoara, Lugoj, Mehadia, and Drobeta is $374 + 120 = 494$)
- Explored set: { Arad, Zerind, Timisoara, Sibiu, Oradea, Rimnicu Vilcea, Lugoj, Fagaras, Mehadia, Pitesti, Drobeta }

13. Expand node Bucharest (418):

- Remove (418, Bucharest) from the priority queue.
- Add Bucharest to the explored set.
- Bucharest is the goal node, and we've reached it via the least costly path.

Summary

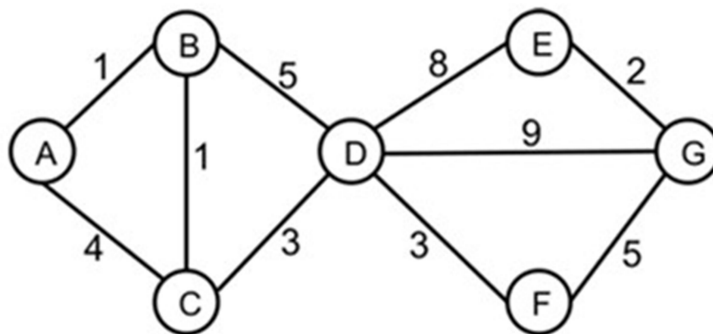
The shortest path from **Arad** to **Bucharest** using UCS is:

- **Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest** with a total cost of **418**.

Conclusion

UCS guarantees the optimal path due to its systematic exploration of nodes by increasing path cost, re-consideration of cheaper paths before node expansion, and avoidance of redundant expansions once nodes are explored.

Example 2: Apply UCS for the below graph, A is start and G is Goal.



Initialization

- Start at node A.
- The priority queue (min-heap) initially contains $(0, A)$ where 0 is the cost to reach A.

Step 1

- Pop $(0, A)$ from the priority queue.
- Expand node A. Its neighbors are B and C.
- Push neighbors of A into the priority queue:

- (1, B) for A → B with cost 1
 - (4, C) for A → C with cost 4
- Priority queue: [(1, B), (4, C)]

Step 2

- Pop (1, B) from the priority queue.
- Expand node B. Its neighbors are A, D, and C.
- Push neighbors of B into the priority queue:
 - (2, C) for B → C with cost 2 (total cost: 1 + 1)
 - (6, D) for B → D with cost 6 (total cost: 1 + 5)
- Since (2, C) is cheaper than (4, C), we will update the path cost to C and remove (4, C).
- Priority queue: [(2, C), (6, D)]

Step 3

- Pop (2, C) from the priority queue.
- Expand node C. Its neighbors are A, B, and D.
- Push neighbors of C into the priority queue:
 - (5, D) for C → D with cost 5 (total cost: 2 + 3)
- Since (5, D) is cheaper than (6, D), we will update the path cost to D and remove (6, D).
- Priority queue: [(5, D)]

Step 4

- Pop (5, D) from the priority queue.
- Expand node D. Its neighbors are B, C, E, F, and G.
- Push neighbors of D into the priority queue:
 - (13, E) for D → E with cost 8 (total cost: 5 + 8)
 - (8, F) for D → F with cost 3 (total cost: 5 + 3)
 - (7, G) for D → G with cost 2 (total cost: 5 + 2)
- Priority queue: [(7, G), (8, F), (13, E)]

Step 5

- Pop (7, G) from the priority queue.
- Goal node G is reached with a cost of 7.

Result

The path found by UCS from A to G is: A → B → D → G with a total cost of 7.

In summary, UCS ensures the least-cost path by always updating the path cost if a cheaper path to a node is found and removing the older, more expensive paths from the priority queue.